

ECE 375: Computer Organization and Assembly Language Programming

Lab 3 – Data Manipulation & the LCD

SECTION OVERVIEW

Complete the following objectives:

- Understand the basics of data manipulation in AVR assembly.
- Learn the basic steps of initializing a program, such as setting up the stack pointer, initializing registers, configuring peripheral devices, etc.
- Use the X, Y, and Z pointers to perform indirect addressing.
- Declare constant data in the program memory, and iteratively move that data into the data memory.
- Use pre-written library functions to interact with a peripheral device.

BACKGROUND

Introduction

For this lab, you will learn to interact with the LCD included on the ATmega32U4 microcontroller board. In order to use the LCD, you will first need to learn how to properly initialize an assembly program. Next, you will learn how to move data from the program memory into the data memory, and then display that data as characters on the LCD.

To help you along the way, a skeleton code has been provided for this lab. Similar to the Lab 1 skeleton C file, this file contains some code that is already written, and some comments indicating where you need to add some code of your own. This skeleton file also provides a good foundation for writing well-structured and well-commented assembly code, as defined in the AVR Assembler Guide.

Initialization

A program initialization (INIT) consists of any code that is only run once, at the beginning of the program. This code does not belong in the MAIN portion of the program, which is why most assembly programs begin with INIT and not

MAIN. INIT is not a function (you wouldn't ever jump to it from within the MAIN part of your program), but rather a collection of instructions that are executed at power on (and at reset). Traditionally, INIT finishes with a jump to MAIN.

There are several things that should be included in the INIT section of a program. The first and foremost is the initialization of the stack pointer. The stack in AVR operates in a higher-to-lower address fashion, meaning that the most recent element placed on the stack is at a lower address than the previous element placed on the stack. Therefore, the stack pointer should initially point to the upper end of the data memory; in other words, to **the location with the highest address in the data memory space**.

In addition to initializing the stack pointer, which **must** be done for your program to perform function calls correctly, there are several other things that are typically initialized within the INIT section of a program, such as:

- I/O ports
- Timer/counters
- Interrupts
- Peripheral devices

The exact contents of INIT depend on the features of the ATmega32U4 board that your program requires. In this lab, you must initialize the LCD, which involves calling the LCDInit subroutine (described later in this document). Additionally, you will need to transfer data from program memory to data memory, so you may want to plan for this operation in your INIT section as well.

Using the LCD Driver

To interact with the LCD, you must use pre-written functions from the LCD driver file provided on the lab webpage (LCDDriver.asm). **Ensure you download this file and place it in the same directory as your AVR assembly code.** Unlike the m32U4def.inc definition file, which only contains pre-compiler directives, the LCD driver includes actual assembly instructions and cannot be included at the start of your main program file. According to the skeleton code, any included code files should be added at the end of the main program, typically in the final line(s). Be aware that the Microchip Studio IDE has a bug that discards the last line of code, so you may need to **add a blank line at the end** to ensure your code is fully included.

After including the LCD driver into your program, you will still need to properly setup and call the functions defined in the LCD driver to interact with the

LCD. (Remember, for any function calls to work correctly, the stack pointer must have already been initialized.) Detailed descriptions of the LCD driver functions are provided in Appendix A at the end of this handout, but here is an overview:

- `LCDInit` – Initialize the LCD (call once from within `INIT`)
- `LCDBacklightOn` – enables the backlight
- `LCDBacklightOff` – disables the backlight
- `LCDWrite` – writes both lines of text to the LCD
- `LCDWrLn1` – writes the top line (16 characters) to the LCD
- `LCDWrLn2` – `LCDWrLn2` - writes the bottom line (16 characters) to the LCD
- `LCDClr` – Clear (write “space” to) all characters of both lines
- `LCDClrLn1` – Clear all characters of line 1 only
- `LCDClrLn2` – Clear all characters of line 2 only
- `LCDWriteByte` – Write directly to a single character of the LCD
- `Bin2ASCII` – Convert an 8-bit unsigned value into an ASCII string

There are other functions within the LCD driver, but they simply support the functions listed above, so they should not be called directly from your program.

Data Manipulation

To transfer data between memory types, it’s essential to understand the organization of the available memory. The ATmega32U4 microcontroller uses an 8-bit AVR architecture, meaning all registers and data memory spaces are 8 bits wide, and data is handled one byte at a time (except for certain instructions that treat specific register pairs as a single 16-bit “word”).

However, the AVR instructions supported by the ATmega32U4 are 16 bits wide (or, for a few instructions, 32 bits). Therefore, for efficiency, the ATmega32U4’s program memory is organized in 16-bit (2-byte) words. This has an important consequence: unlike pointers to data memory, pointers to program memory **initially point to a 16-bit memory location**. To access just the low byte or the high byte of that location, specific steps are required.

When writing and testing your program, it can be helpful to include data directly in program memory. For instance, if you are simulating your program and testing a function that uses several data memory locations as input, manually entering test values via the Memory window in Microchip Studio may be cumbersome. Instead, you can use the `.DB` (Data Byte) directive to instruct the

```

• Z <- program memory address of first character
• Y <- data memory address of character destination
• do { mpr <- ProgramMemory[Z], Z++,
      DataMemory[Y] <- mpr, Y++ }
      while (Z != program memory address after last character)

```

Figure 1: Pseudocode for String Copy from Program Memory to Data Memory

compiler to place test values into program memory during compilation. You can then write a simple loop or function to copy these values from program memory to data memory at runtime.

The following example shows how to place data into the program memory:

```

DATA:
    .DB $05, $F4

```

The hexadecimal values \$05 and \$F4 can then be accessed using the label `DATA`, which the compiler converts to a program memory address. To read this data, use the `LPM` (Load Program Memory) instruction. You can also enter strings (i.e., a sequence of ASCII characters) into program memory using the `.DB` directive; just take a look at the Lab 3 skeleton file to see how.

Note: Since the program memory is 16 bits wide, the total amount of data you specify **using a single instance** of the `.DB` directive should be an integer multiple of 16 bits. Otherwise, the compiler will pad the program memory with an extra byte of data, usually \$FF.

Movement within data memory is accomplished using variations of load and store instructions, and with two main addressing modes: *direct addressing* and *indirect addressing*. Direct addressing, where a memory address is provided directly as an operand, is useful when you want to move a single byte to/from a single memory location (such as an extended I/O register). Indirect addressing, where the X, Y, and Z-pointers are an operand and they *contain an effective address*, is useful for moving blocks of data into contiguous locations in the data memory. By using indirect addressing in conjunction with a loop, you can efficiently move lots of data around in memory. Figure 1 shows some pseudocode you can use to properly read a bunch of data from program memory.

PROCEDURE

The display should be initially blank. To receive full credit, you must: (1) properly initialize your program, (2) place your two strings into program memory using the `.DB` directive. (3) Use two unique strings, for example one string could contain your name, and another your partner's. (4) copy the strings from program memory to the appropriate data memory locations **using a loop**, and (5) there must not be any unintended trailing characters on the LCD.

You will use three buttons; PD4, PD5, and PD7 to write and clear text on the LCD display. Write a program that does the following:

1. When you press PD4: The content should be cleared.
2. When you press PD5: Displays your name on the first line of the LCD, and a phrase like "Hello World!" (or your partner's name) on the second line of the screen.
3. When you press PD7: To make the display more dynamic, modify your program to show a scrolling, marquee-style message across the two lines of the LCD, moving from left to right. Add a delay between each "scroll" event to allow enough time for the characters to be read in their current position before they shift. When a character reaches the end of a line, it should wrap around to the beginning of **the opposite line**. For instance, a character at index 15 of line 1 should move to index 0 of line 2, and a character at index 15 of line 2 should move to index 0 of line 1.

Button PD4: Line 1:
 Line 2:

Button PD5: Line 1: Your name
 Line 2: Hello, World (or partner's name)

Button PD7:

- 1) Line 1: ____My_Name_is_
 Line 2: _____Jane_Doe_
 (wait for .25 seconds)
- 2) Line 1: _____My_Name_is
 Line 2: _____Jane_Doe
 (wait for .25 seconds)
- 3) Line 1: e_____My_Name_i
 Line 2: s_____Jane_Do
 (wait for .25 seconds)

4) Line 1: oe_____My_Name_
 Line 2: is_____Jane_D
 etc.

LAB REPORT

You are **not** required to write a lab report, but you should review the study questions. Although these questions will not be graded, they **can be asked during your demo, and your grade may be affected** if you cannot answer them.

Study Questions

1. In this lab, you were required to move data between two memory types: program memory and data memory. Explain the intended uses and key differences of these two memory types.
2. You also learned how to make function calls. Explain how making a function call works (including its connection to the stack), and explain why a `RET` instruction must be used to return from a function.
3. Write pseudocode for an 8-bit AVR function that will take two 16-bit numbers (from data memory addresses \$0111:\$0110 and \$0121:\$0120), **add them together**, and then store the 16-bit result (in data memory addresses \$0101:\$0100). (Note: The syntax "\$0111:\$0110" is meant to specify that the function will expect *little-endian* data, where the highest byte of a multi-byte value is stored in the highest address of its range of addresses.)
4. Write pseudocode for an 8-bit AVR function that will take the 16-bit number in \$0111:\$0110, **subtract it from** the 16-bit number in \$0121:\$0120, and then store the 16-bit result into \$0101:\$0100.

APPENDIX A – LCD FUNCTIONS

LCDInit

This subroutine initializes the serial interface that is used to communicate with the LCD, and also initializes the display itself, configuring it to display 2 lines/rows of 16 characters (2x16). This function can be called directly, e.g.:

```
rcall LCDInit ; Initialize LCD peripheral interface
```

LCDWrite

This function writes data to both lines of the LCD. First, line data is retrieved from the following data memory addresses:

Line 1: \$0100 – \$010F

Line 2: \$0110 – \$011F

Next, that data is actually written out to the LCD. So, for this function to work properly, **you must first move the data** (e.g., an ASCII string) that you want displayed on the LCD into the appropriate locations in data memory. Then, call the function as follows:

```
rcall LCDWrite ; Write to both lines of the LCD
```

LCDWrLn1

This function writes data to the first/top line of the LCD. First, the data is retrieved from data memory addresses \$0100 – \$010F, and then the data is written out to the first line of the LCD.

To use this function, make sure the data you want to display is in the appropriate locations in data memory, and then call the function as follows:

```
rcall LCDWrLn1 ; Write to first line (Line 1) of the LCD
```

LCDWrLn2

This function writes data to the second/bottom line of the LCD. First, the data is retrieved from data memory addresses \$0110 – \$011F, and then the data is written out to the second line of the LCD.

To use this function, make sure the data you want to display is in the appropriate locations in data memory, and then call the function as follows:

```
rcall LCDWrLn2 ; Write to second line (Line 2) of the LCD
```

LCDClr

This subroutine clears both lines of the LCD. Specifically, it writes the “space” character (ASCII value \$20) to data memory locations \$0100 – \$010F and \$0110 – \$011F, and then writes to both lines of the LCD. To call it, use:

```
rcall LCDClr ; Clear both lines of the LCD
```

LCDClrLn1

This subroutine works similarly to LCDClr, except it only clears the first/top line of the LCD.

```
rcall LCDClrLn1 ; Clear first line (Line 1) of the LCD
```

LCDClrLn2

This subroutine works similarly to LCDClr, except it only clears the second/bottom line of the LCD.

```
rcall LCDClrLn2 ; Clear second line (Line 2) of the LCD
```

LCDWriteByte

This function allows you to write a single ASCII character (or byte) to anywhere on the LCD. It allows direct control over where data is displayed on the LCD, and does not require anything to be stored in data memory first (unlike many of the previous functions). There are three registers that must be loaded with input parameters before this function is called:

- **mpr** – Contains the byte/character that you want to display on the LCD (must be a value between 0 and 255)
- **line** – Contains the line number where you want to display the byte (must be either 1 for first line, or 2 for second line))
- **count** – Contains the index number where you want to display the byte (must be a number between 0 and 15, with 0 specifying the leftmost position). Note: The LCD actually accepts index values up to 39, but only indexes 0 through 15 are actually visible on the display. Any index value between 16 and 39 will result in the character being display “off screen”

The following example uses the `LCDWriteByte` function to write the character 'D' to Line 2, index 7 of the LCD:

```
; load parameters for LCDWriteByte
ldi    mpr, 'D'      ; load character 'D' into mpr
ldi    line, 2       ; load line number 2 into line register
ldi    count, 7      ; load index 7 into count register

; call LCDWriteByte function
rcall  LCDWriteByte ; write character to LCD
```

Bin2ASCII

This function converts an unsigned 8-bit value into its numerically equivalent ASCII string, i.e. from $138_{10} \rightarrow$ "138". The resulting string (with length between 1 and 3 bytes) is stored in the data memory. This function is useful for displaying a value that changes throughout the course of a program (such as a counter).

Two registers must be loaded with input parameters before this function is called, and a third register must be available to store a returned value:

- `mpr` – Contains the 8-bit value that will be converted
- `XH:XL` – Contains the beginning 16-bit data memory address where the ASCII string will be stored
- `count` – Will contain the return value (the length of the created string)

The following example uses the `Bin2ASCII` function to convert the value 138_{10} into "138", and then store it in data memory beginning at location `$0112`:

```
; load parameters for Bin2ASCII
ldi    mpr, 138      ; load to-be-converted value into mpr
ldi    XL, low($0112) ; load X with beginning address
ldi    XH, high($0112) ; of where result will be stored

; call Bin2ASCII function
rcall  Bin2ASCII      ; convert value in ASCII
```

After the above example is executed, the data memory locations `$0112`, `$0113`, and `$0114` will contain the characters '1', '3', and '8', respectively, and `count` will contain the value 3.

If a smaller value like 75_{10} had instead been converted, only locations `$0112` and `$0113` would contain characters, and `count` would have a value of 2.